



COSMOS AI at Scale

Recurrent Neural Network and Transformer

July 2026, Instructor: Meng Tang

Recap: A bigram model

- Alice was beginning to get very tired of sitting by her sister on the bank, and of having nothing to do: once or twice she had peeped into the book her sister was reading, but it had no pictures or conversations in it, 'and what is the use of a book,' thought Alice 'without pictures or conversation?'

$$P(w^{(i)} = \text{of} \mid w^{(i-1)} = \text{tired}) = 1$$

$$P(w^{(i)} = \text{of} \mid w^{(i-1)} = \text{use}) = 1$$

$$P(w^{(i)} = \text{sister} \mid w^{(i-1)} = \text{her}) = 1$$

$$P(w^{(i)} = \text{beginning} \mid w^{(i-1)} = \text{was}) = 1/2$$

$$P(w^{(i)} = \text{reading} \mid w^{(i-1)} = \text{was}) = 1/2$$

$$P(w^{(i)} = \text{bank} \mid w^{(i-1)} = \text{the}) = 1/3$$

$$P(w^{(i)} = \text{book} \mid w^{(i-1)} = \text{the}) = 1/3$$

$$P(w^{(i)} = \text{use} \mid w^{(i-1)} = \text{the}) = 1/3$$

Our first neural net for NLP:

A neural n -gram model

Given a fixed-size vocabulary V , an n -gram model predicts the probability of the n -th word following the preceding $n-1$ words:

$$P(w^{(i)} | w^{(i-1)}, w^{(i-2)}, \dots, w^{i-(n-1)})$$

How can we model this with a neural net?

- **Input layer:** concatenate $n-1$ word vectors
- **Output layer:** a softmax over $|V|$ units

An n-gram model $P(w \mid w_1 \dots w_k)$ as a feedforward net (**naively**)

Assumptions:

The **vocabulary** V contains V types (incl. UNK, BOS, EOS)
We want to condition each word on k preceding words

Our (naive) model:

— [Naive]

Each **input word** $w_i \in V$ is a **V -dimensional one-hot vector** $v(w)$

→ The **input layer** $\mathbf{x} = [v(w_1), \dots, v(w_k)]$ has **$V \times k$ elements**

— We assume **one hidden layer** $\mathbf{h}$

— The **output layer** is a softmax over V elements

$$P(w \mid w_1 \dots w_k) = \text{softmax}(\mathbf{h}\mathbf{W}^2 + \mathbf{b}^2)$$

An n -gram model $P(w \mid w_1 \dots w_k)$ as a feedforward net (**better**)

Assumptions:

The **vocabulary** V contains V types (incl. UNK, BOS, EOS)
We want to condition each word on k preceding words

Our (better) model:

— [Better]

Each **input word** $w_i \in V$ is an **n -dimensional dense embedding vector** $v(w)$ (with $n \ll V$)

→ The **input layer** $\mathbf{x} = [v(w_1), \dots, v(w_k)]$ has **$n \times k$ elements**

— We assume **one hidden layer** $\mathbf{h}$

— The **output layer** is a softmax over V elements

$$P(w \mid w_1 \dots w_k) = \text{softmax}(\mathbf{h}\mathbf{W}^2 + \mathbf{b}^2)$$

1D CNNs for text

Text is a (variable-length) **sequence** of words (word vectors)

[#channels = dimensionality of word vectors]

We can use a **1D CNN** to slide a window of n tokens across:

— Filter size $n = 3$, stride = 1, no padding

The quick brown fox jumps over the lazy dog

The **quick brown fox** jumps over the lazy dog

The quick **brown fox jumps** over the lazy dog

The quick brown **fox jumps over** the lazy dog

The quick brown fox **jumps over the** lazy dog

The quick brown fox jumps **over the lazy** dog

— Filter size $n = 2$, stride = 2, no padding:

The quick brown fox jumps over the lazy dog

The quick **brown fox** jumps over the lazy dog

The quick brown fox **jumps over** the lazy dog

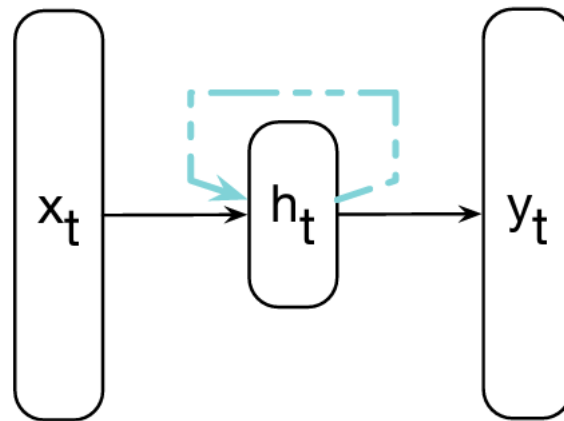
The quick brown fox jumps over **the lazy** dog

Outline

- ❑ Recurrent Neural Network
- ❑ RNNs as Language Models
- ❑ RNNs for other NLP tasks
- ❑ Staked and Bi-directional RNN
- ❑ Encoder-decoder model

Recurrent Neural Network

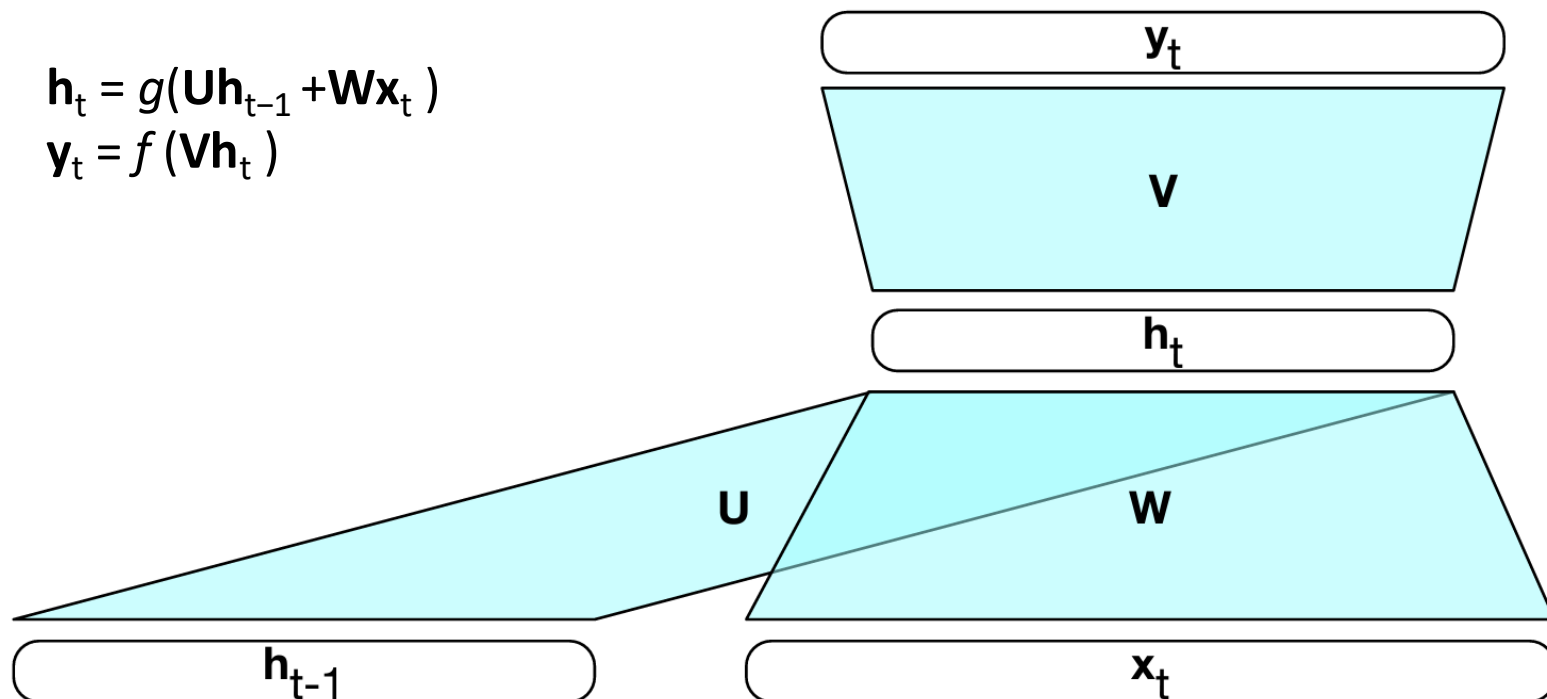
- ❑ Temporal nature in language processing
- ❑ RNN deals with sequential input data stream like language.



A simple RNN

A Simple Recurrent Neural Network

□ RNN illustrated as a feed-forward network

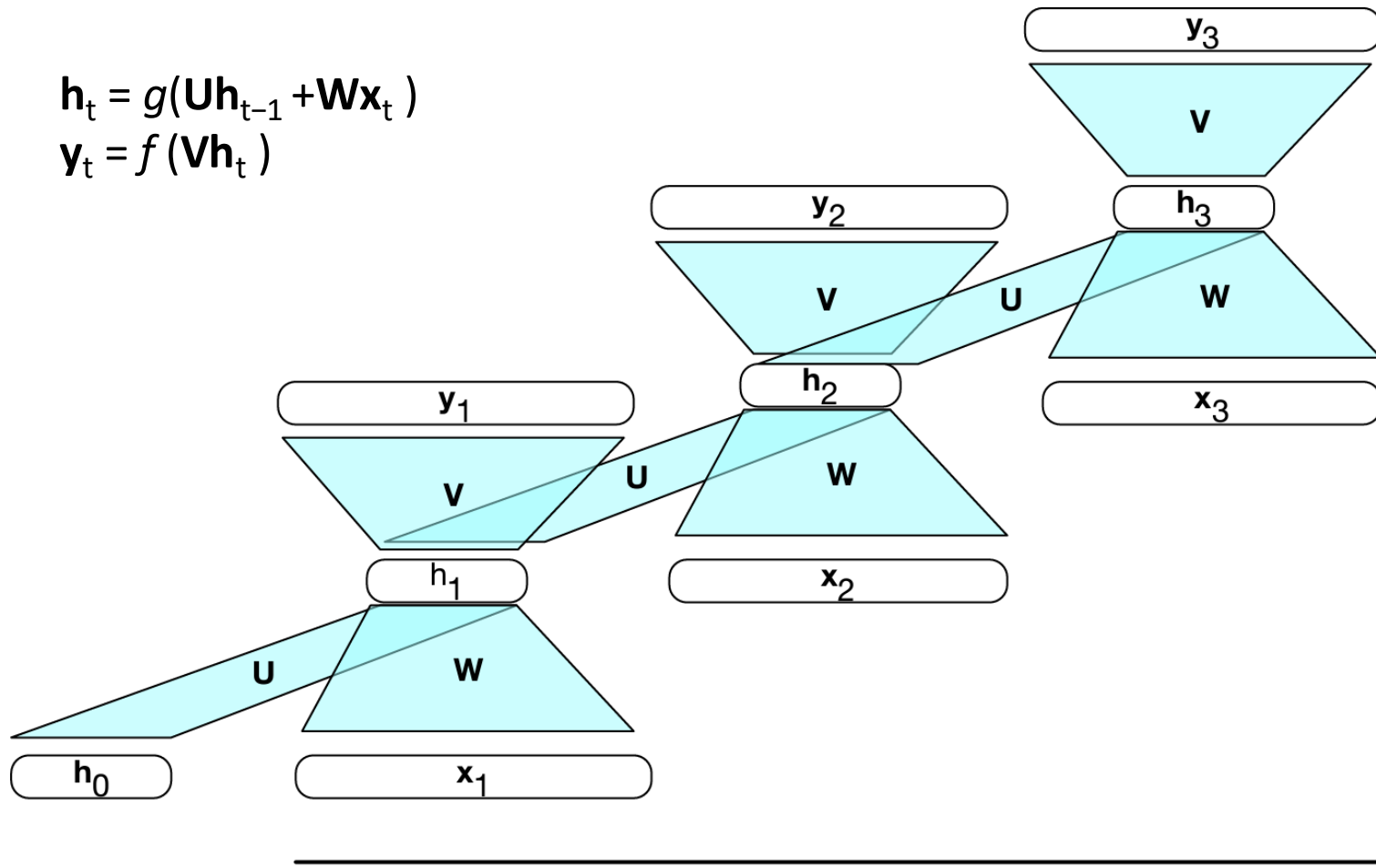


A Simple Recurrent Neural Network

□ RNN unrolled in time

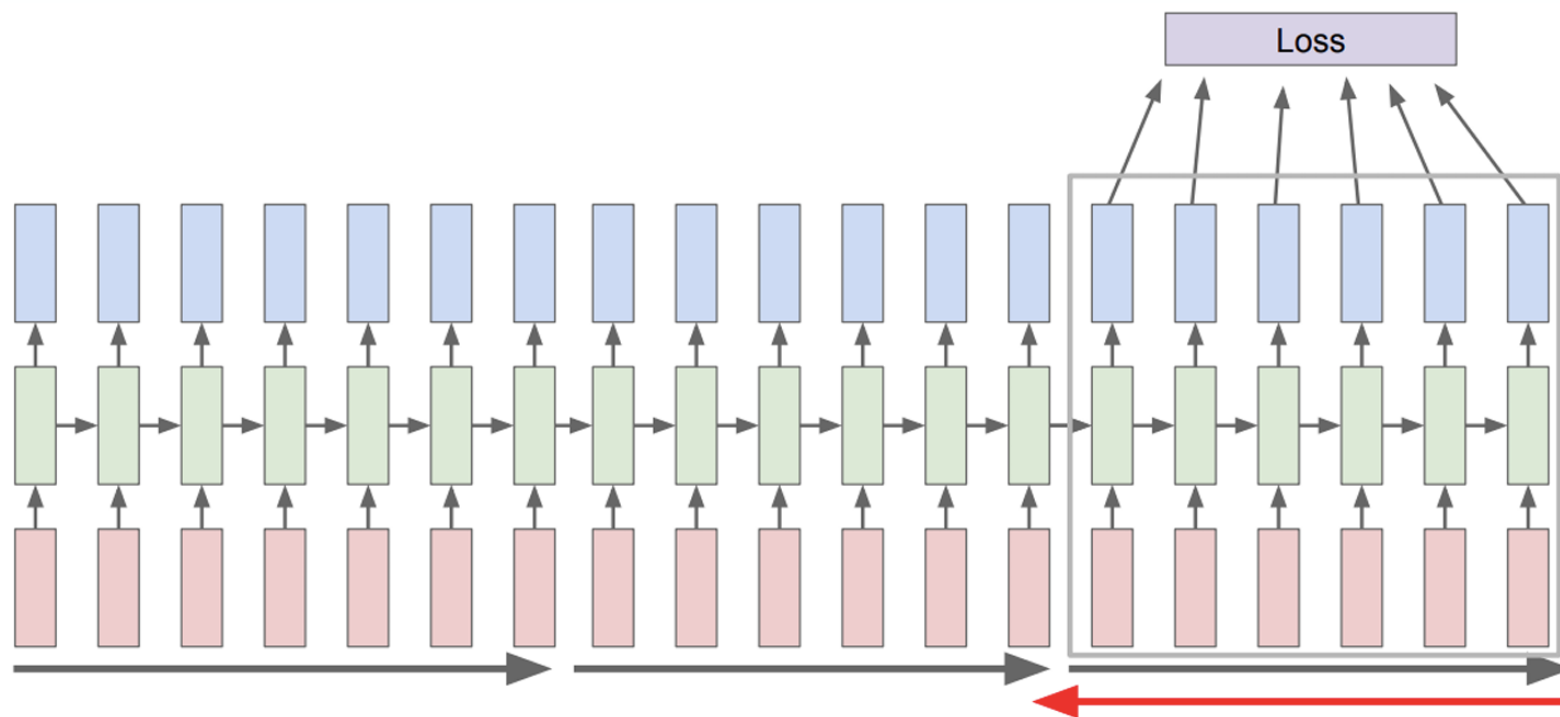
$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$

$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$



Truncated backpropagation through time

- ❑ Backpropagation is very expensive for long sequences



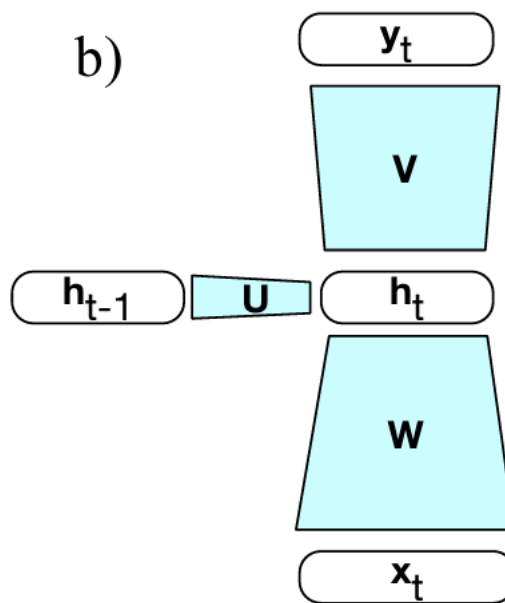
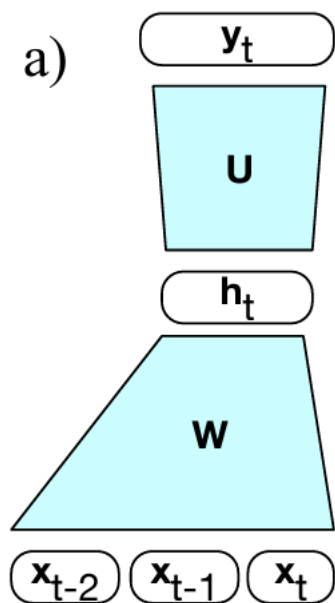
- ❑ Run forward and backward through chunks of the sequence instead of whole sequence

RNNs as Language Models

- Language models predict the next word in a sequence given some preceding context.

$P(\text{fish} | \text{Thanks for all the})$

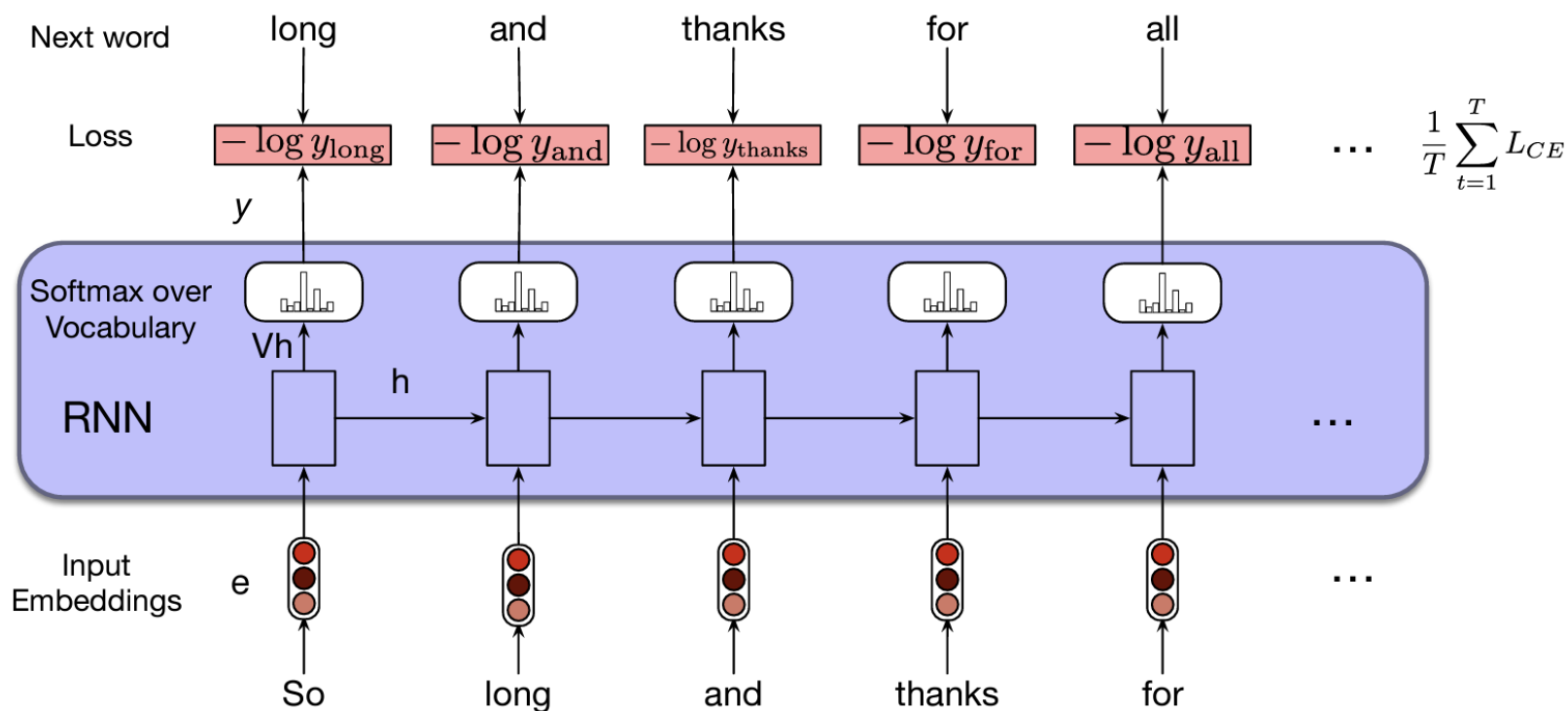
- RNN Language Model



$$\begin{aligned} \mathbf{e}_t &= \mathbf{E}\mathbf{x}_t \\ \mathbf{h}_t &= g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \\ \mathbf{y}_t &= \text{softmax}(\mathbf{V}\mathbf{h}_t) \end{aligned}$$

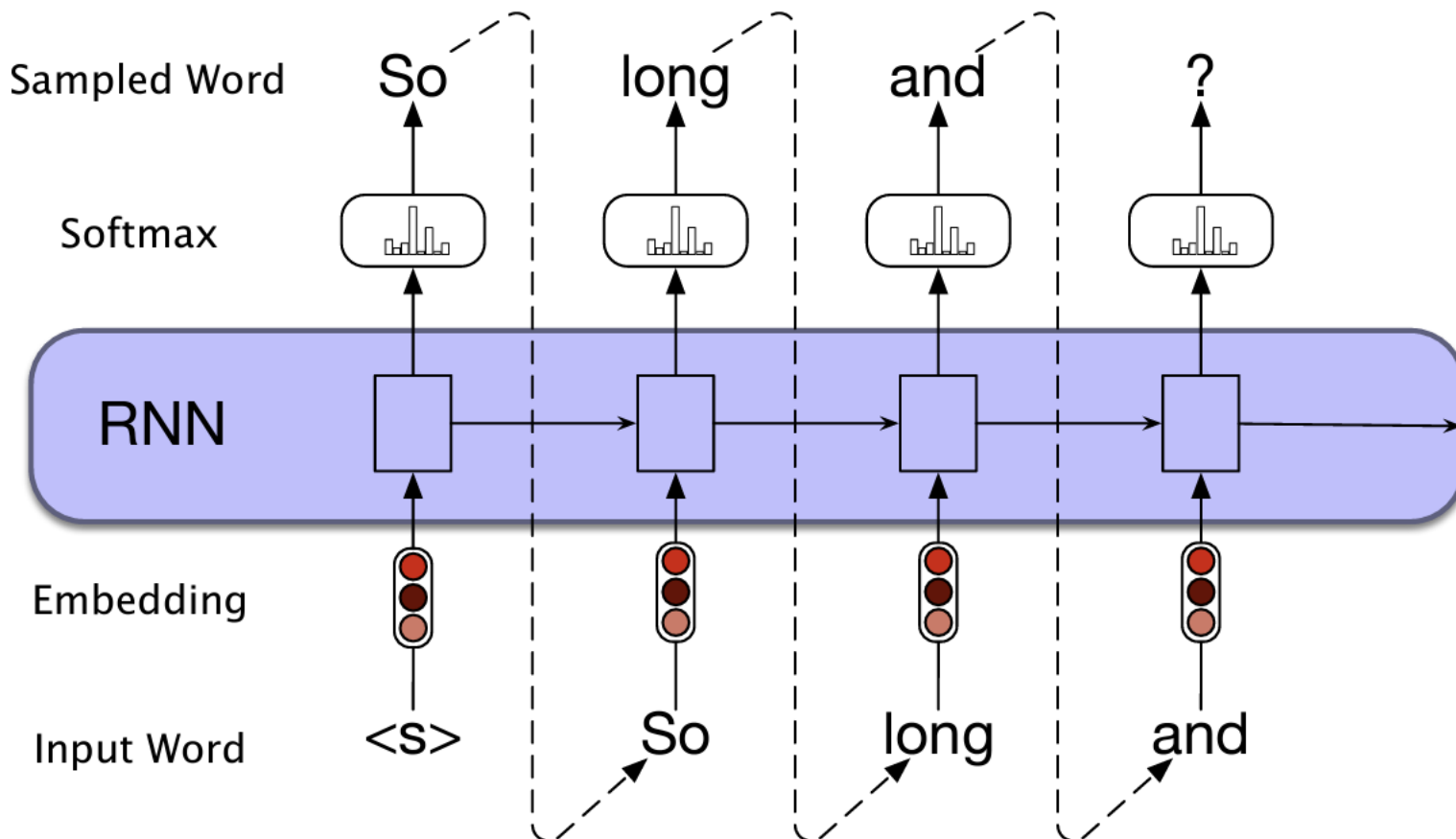
Training an RNN Language Model

Maximum likelihood estimation



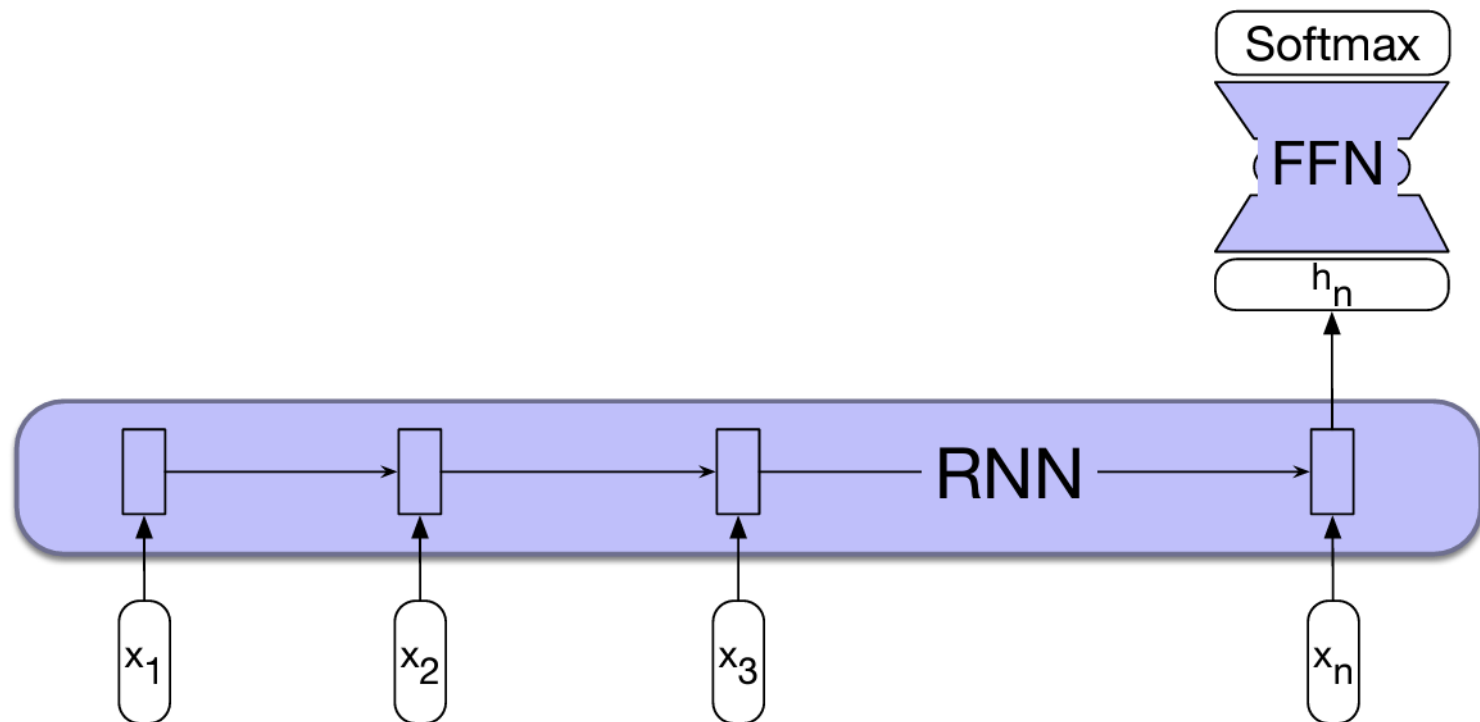
Generation with RNN Language Model

- ❑ Autoregressive (casual) generation
- ❑ Unlike teacher forcing during training



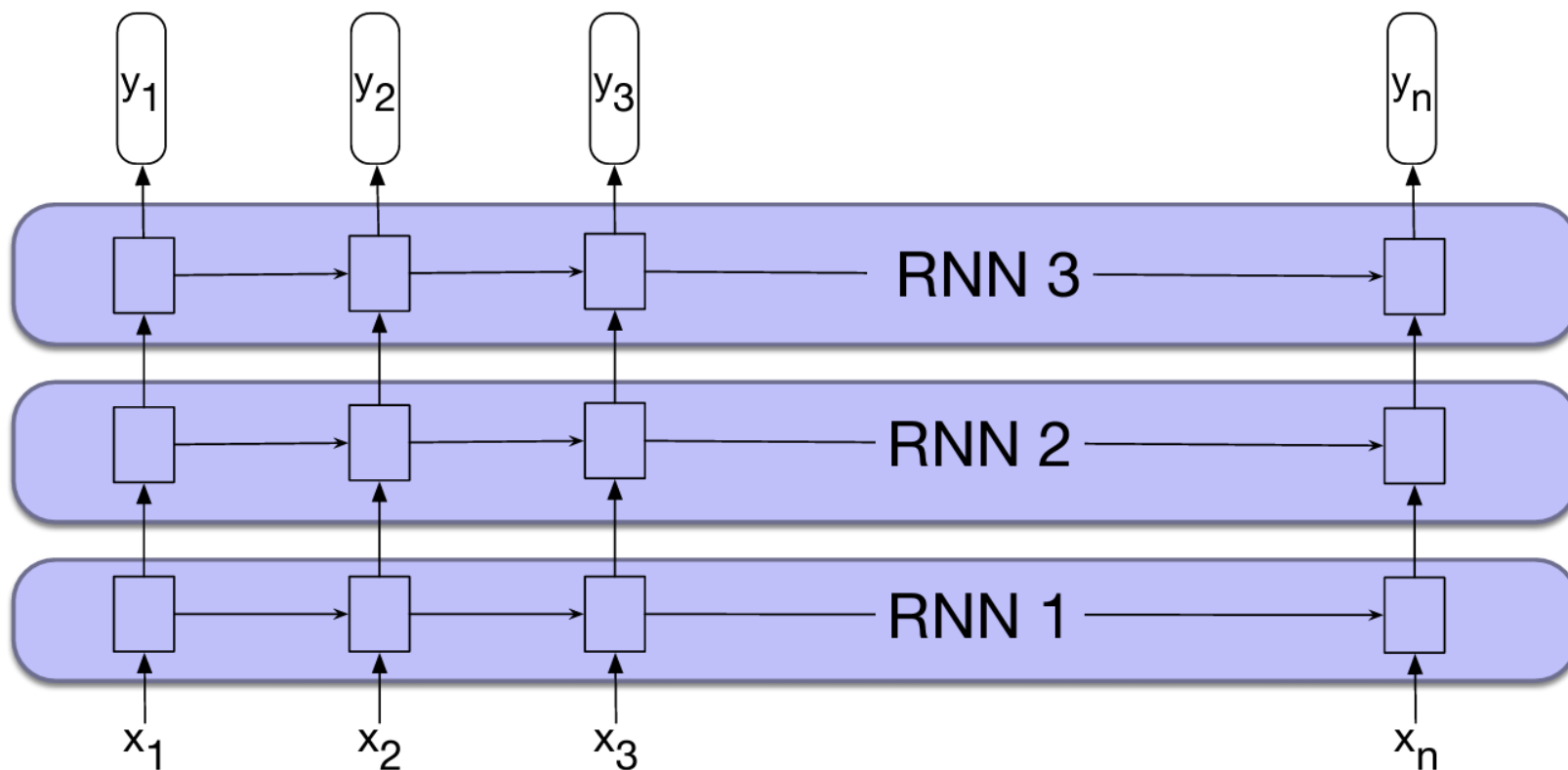
RNNs for other NLP tasks

- ❑ RNN for sequence classification
- ❑ Commonly called text classification, like sentiment analysis or spam detection



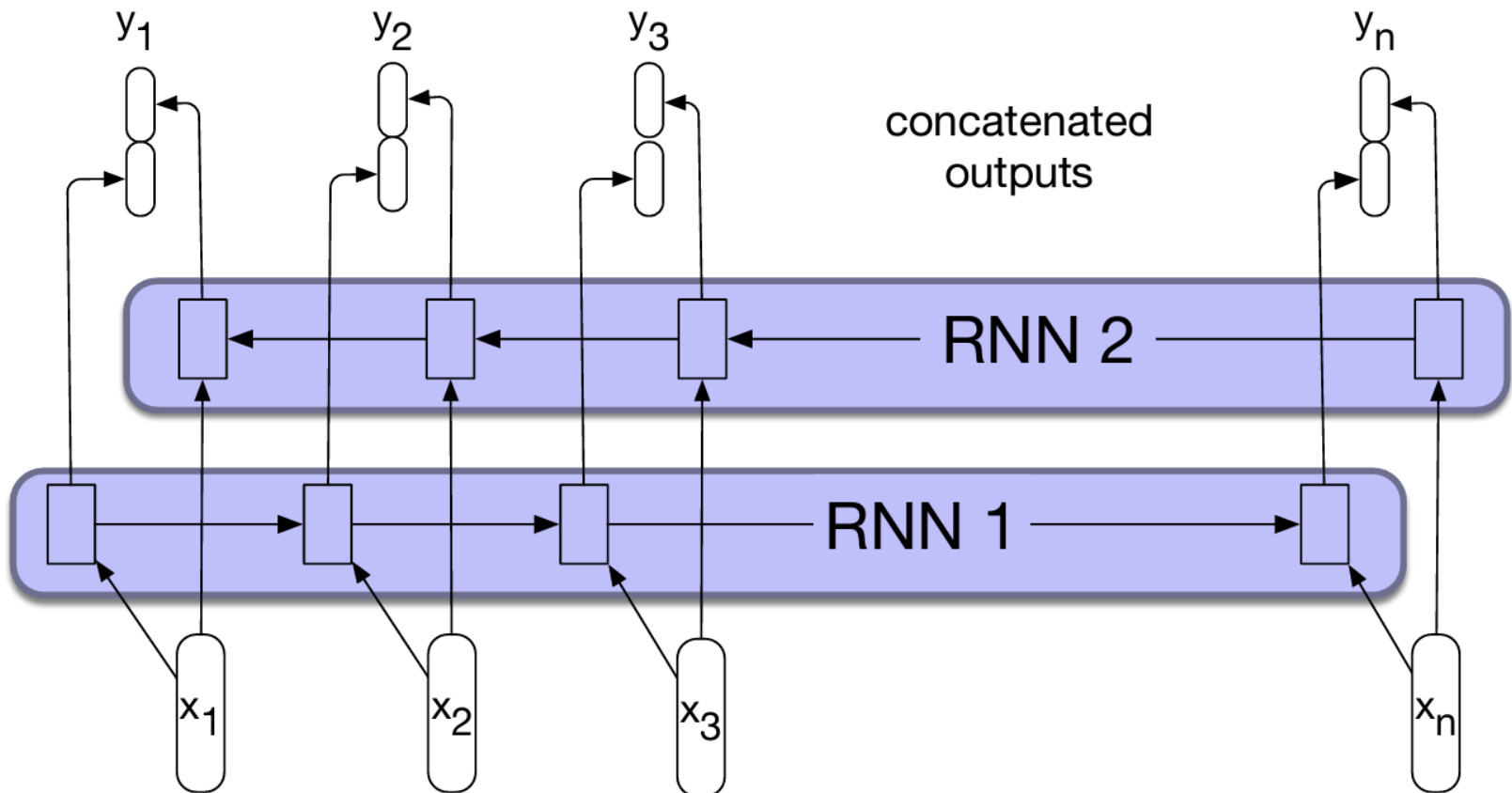
Stacked RNN

□ Outputs from one RNN as an input sequence to another one



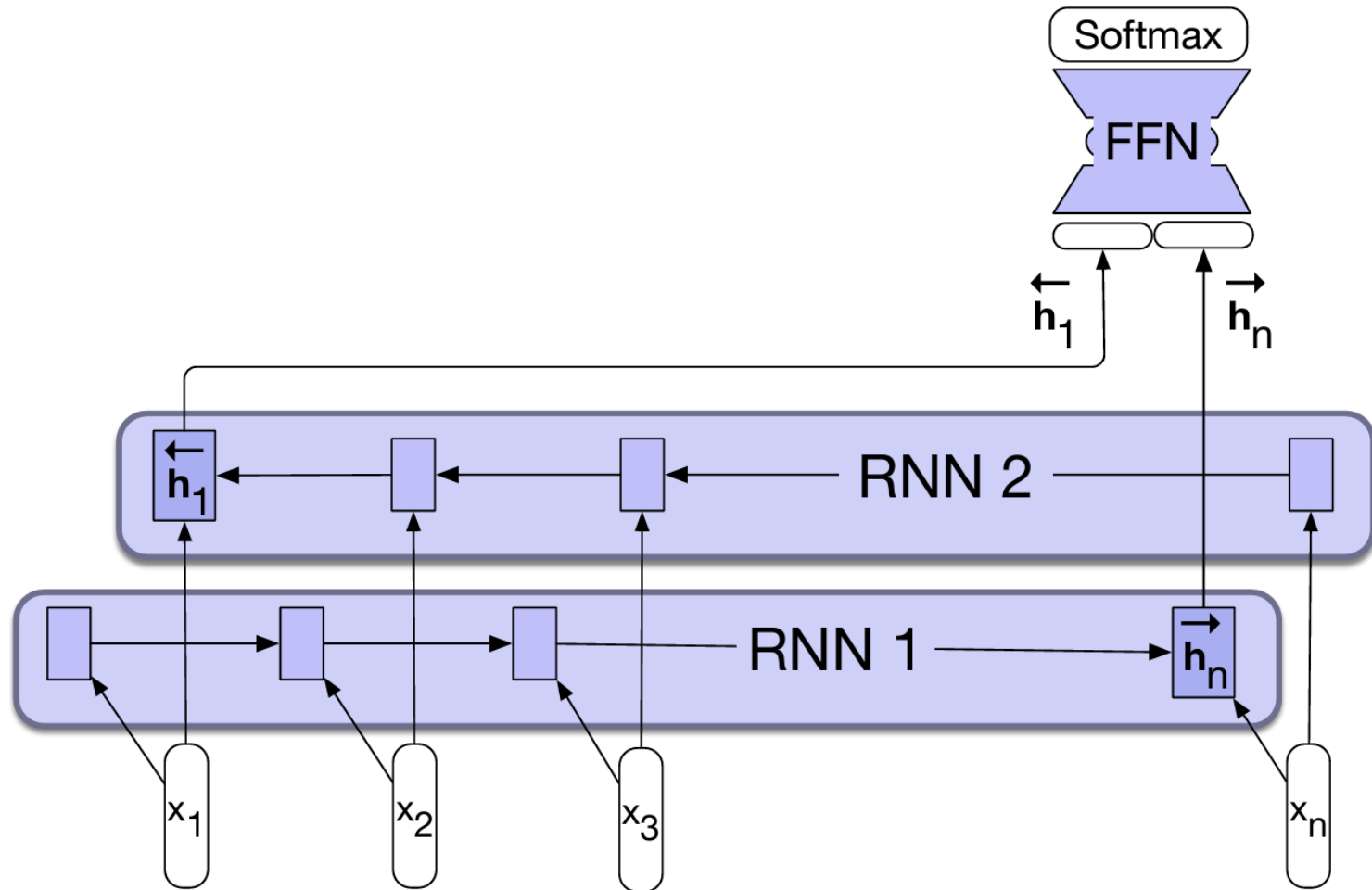
Bidirectional RNN

- Two independent RNN, one forward, another backward



Bidirectional RNN for sequence classification

- Combine hidden states from first & last word



Application of Bidirectional RNN

- ❑ Suppose we want to tag each word in a sentence with its part of speech.

“He said it was cold.”

- ❑ To correctly tag the word “was,” we need both:
 - ❑ **past context** (“He said it”) → suggests a verb is coming
 - ❑ **future context** (“cold”) → indicates that “was” is a **linking verb**

Application of Bidirectional RNN

- ❑ In **automatic speech recognition (ASR)**, the goal is to convert an audio sequence into text.
- ❑ A unidirectional RNN processes audio left-to-right and can only use **past information**.
- ❑ A **bidirectional RNN** processes the same sequence **both left-to-right and right-to-left**, so it can understand **coarticulation** effects — how the pronunciation of one sound depends on surrounding sounds.

Example Implementation (PyTorch)

python

 Copy code

```
import torch
import torch.nn as nn

# Input: batch_size=1, seq_len=5, input_size=10
x = torch.randn(1, 5, 10)

# Bidirectional RNN
rnn = nn.RNN(input_size=10, hidden_size=20, num_layers=1, bidirectional=True)

output, hidden = rnn(x)

print("Output shape:", output.shape) # (1, 5, 40) since 20*2 directions
```

Vanishing/exploding gradients

- Consider the gradient of L_t at step t , with respect to the hidden state $\mathbf{h}_k$ at some previous step k ($k < t$):

$$\frac{\partial L_t}{\partial \mathbf{h}_k} = \frac{\partial L_t}{\partial \mathbf{h}_t} \left(\prod_{t \geq j > k} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right)$$

- Recurrent multiplication
- Gradients too small (vanishing gradient) or too large (exploding gradient)

Exploding gradients

- ❑ What is the problem?
- ❑ We take a very large step in SGD
- ❑ Solution: Gradient clipping

Algorithm 1 Pseudo-code for norm clipping

```
 $\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$   
if  $\|\hat{\mathbf{g}}\| \geq threshold$  then  
     $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$   
end if
```

Vanishing gradients

- ❑ What is the problem?
- ❑ Parameters barely get updated (no learning)

Residual learning

- ❑ Most cited paper in computer vision and machine learning

Deep Residual Learning for Image Recognition

297720

2016

K He, X Zhang, S Ren, J Sun

Computer Vision and Pattern Recognition (CVPR), 2016

- ❑ A residual connection means adding the input of a layer directly to its output:

$$y = F(x) + x$$

Why residual connection?

$$y = F(x) + x$$

- ❑ Better Gradient Flow
 - ❑ Directly address vanishing gradient
 - ❑ Indirectly address exploding gradient
- ❑ Reuse early layer features
- ❑ Easier Optimization
- ❑ Allowing **very deep networks (100+ layers)**

Local vs distant information

- ❑ Hidden states tend to contain local information

- ❑ But distant information is critical

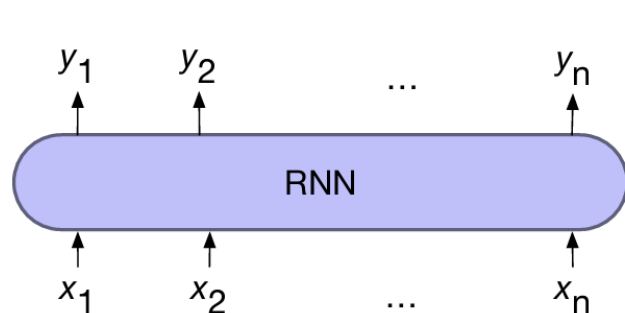
“The flights the airline was canceling were full”

- ❑ Should predict “were” given distant information (flights)

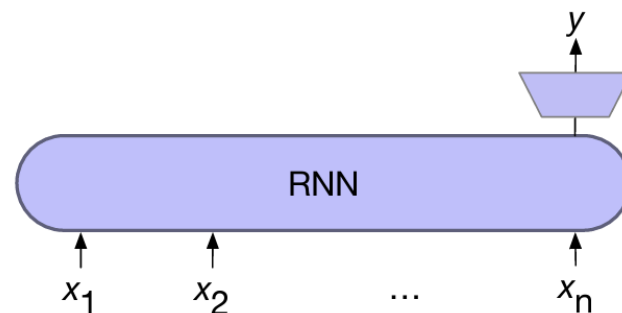
Residual in PyTorch

```
class ResidualBlock(nn.Module):  
    def __init__(self, dim):  
        super().__init__()  
        self.block = nn.Sequential(  
            nn.Linear(dim, dim),  
            nn.ReLU(),  
            nn.Linear(dim, dim)  
        )  
    def forward(self, x):  
        return self.block(x) + x  # residual connection
```

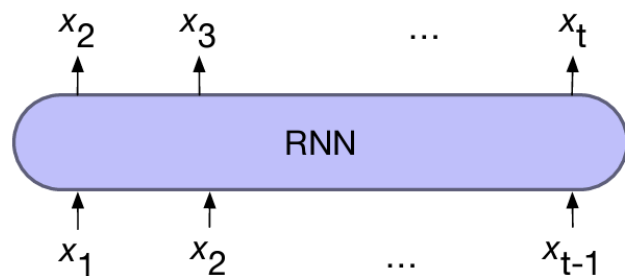
Summary: Common RNN NLP architectures



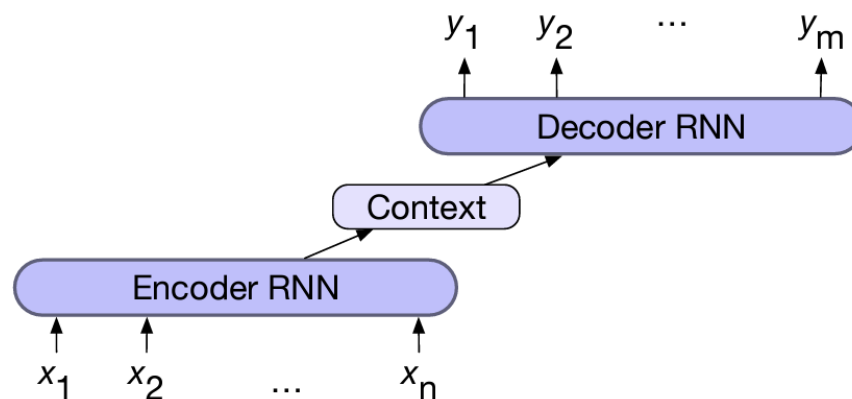
a) sequence labeling



b) sequence classification



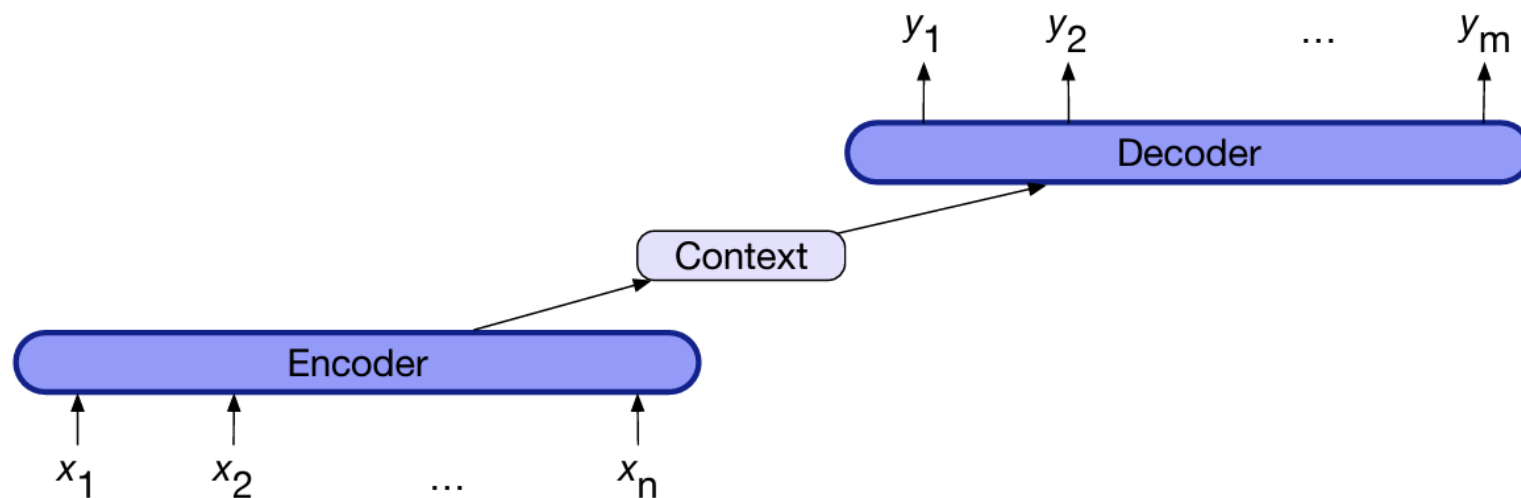
c) language modeling



d) encoder-decoder

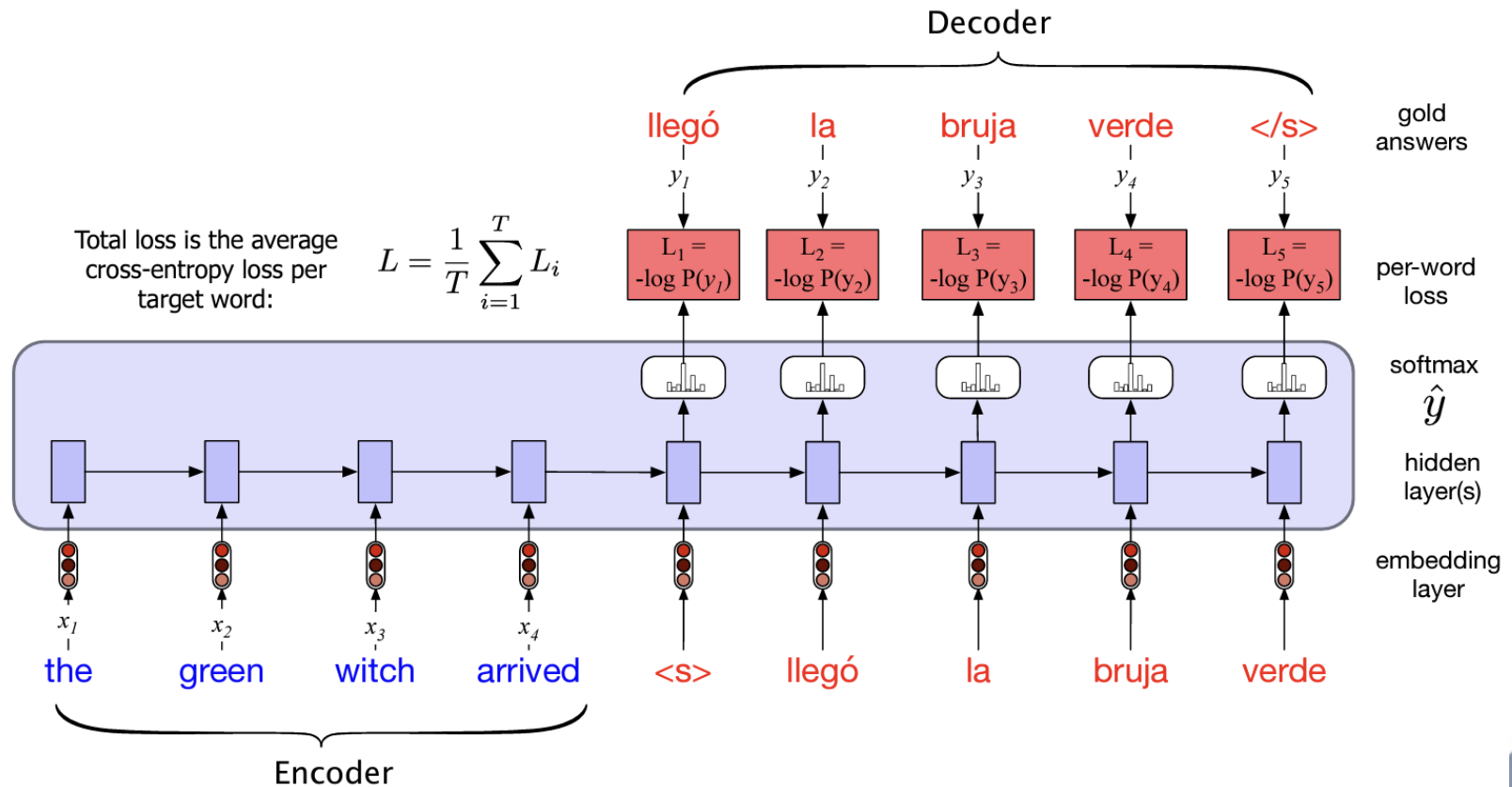
Encoder Decoder Architecture

- ❑ Arbitrary length output given an input sequence
- ❑ A.K.A. sequence-to-sequence network
- ❑ Context vector conveys the essence of the input to the decoder



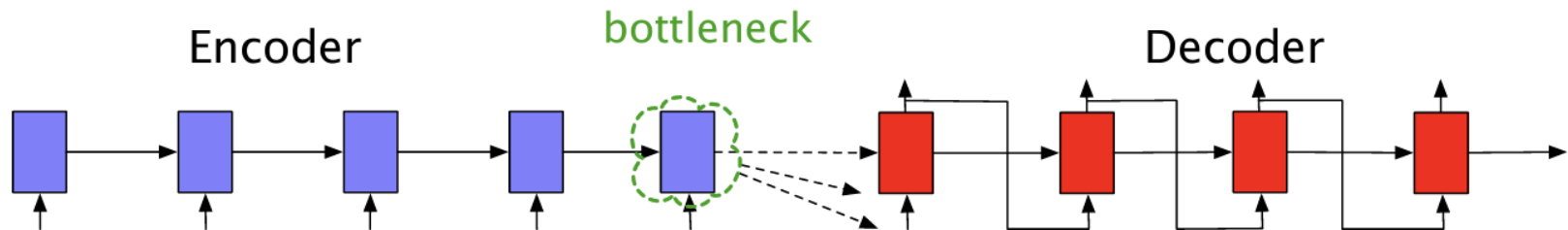
Encoder Decoder Architecture

□ Training an encoder-decoder for machine translation



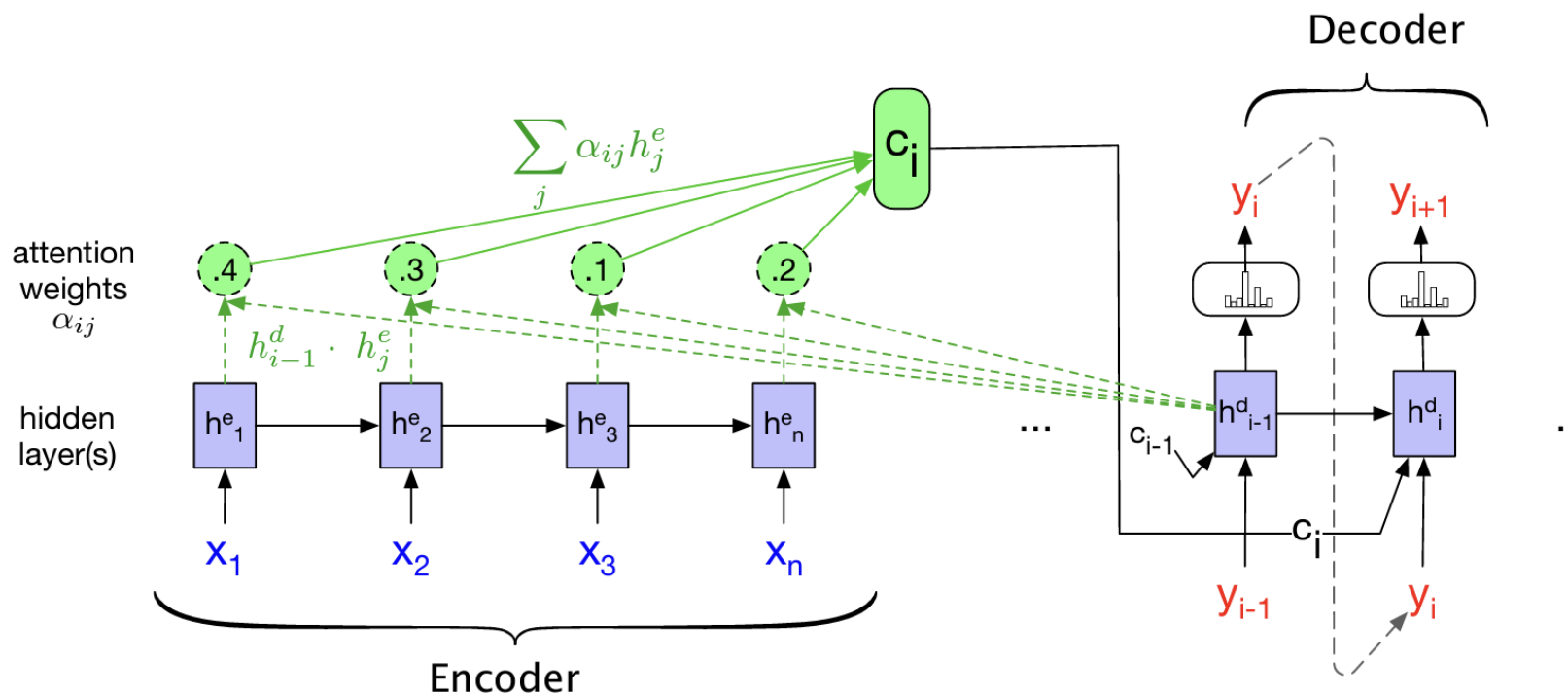
Problem of Encoder-decoder architecture

- ❑ Context vector encodes EVERYTHING about input sequence
- ❑ Context vector acts as a bottleneck



Attention Mechanism

- ❑ Each output in decoder accesses all the hidden states from the encoder, not just the last state
- ❑ Each output attends to all input



What's next?

- ☐ Self-attention
- ☐ Transformer block
- ☐ Transformer architecture
- ☐ Large Language Model
- ☐ Vision Transformers

Transformer: The intuition

- ❑ Context matters for natural language understanding

- ❑ For example:

- ❑ The **chicken** crossed the road because **it** wanted to get to the other side

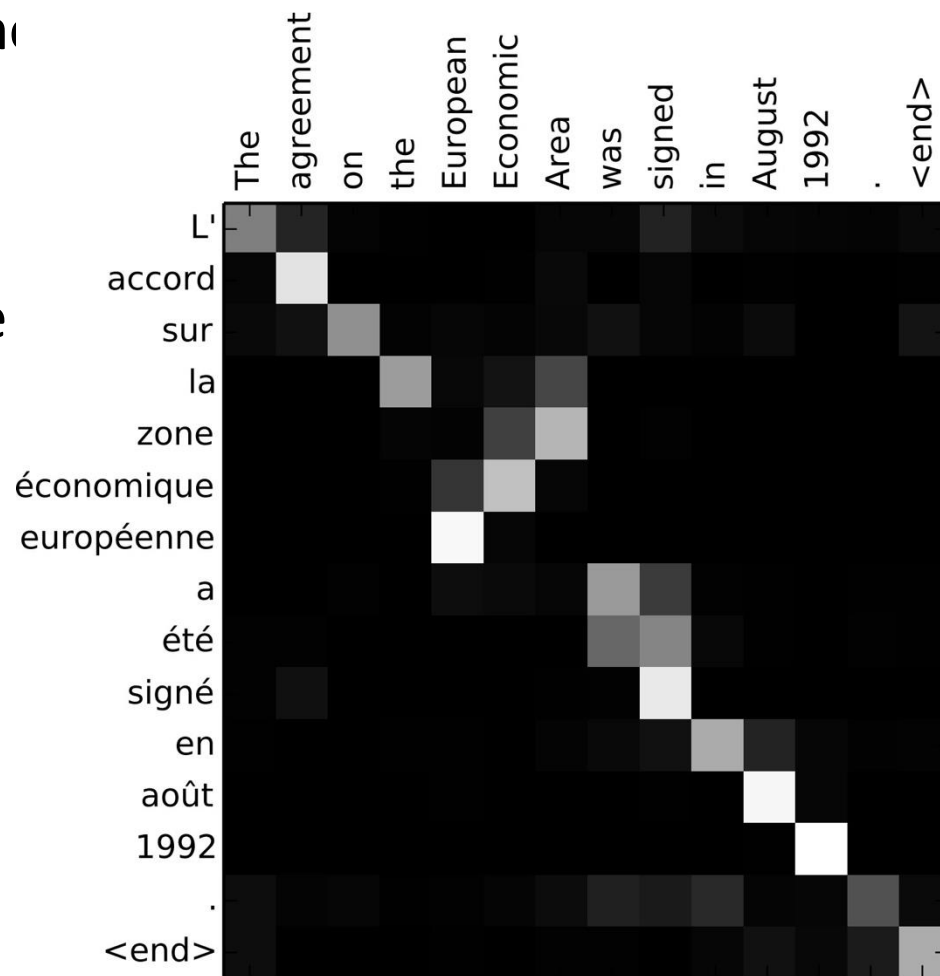
- ❑ I walked along the **pond**, and noticed that one of the trees along the **bank** had fallen into the **water** after the storm.

Attention weights between words

❑ Example: English to French translation

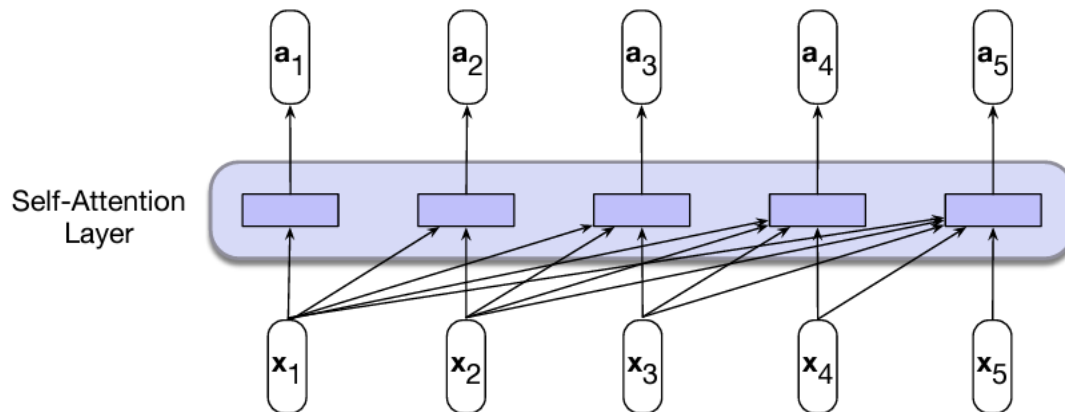
❑ Input: “The agreement on the European Economic Area was signed in August 1992.”

❑ Output: “L'accord sur la zone économique européenne a été signé en août 1992.”



Casual or backward-looking self-attention

- Attends to all the inputs up to, and including, the current one



Self-attention

□ Version 1:

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\begin{aligned}\alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i \\ &= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^i \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \quad \forall j \leq i\end{aligned}$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

Query, Key, and Value

- ❑ Query: the current focus of attention when being compared to all of the other preceding inputs.
- ❑ Key: a preceding input being compared to the current focus
- ❑ Value: used to compute the output for the current focus

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

- ❑ Version 2:

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{q}_i \cdot \mathbf{k}_j$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Self-attention

□ Final Version

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

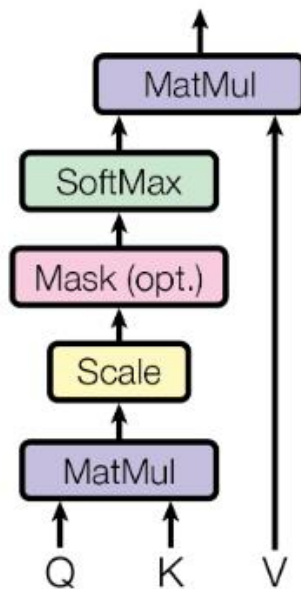
$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Attention Operation

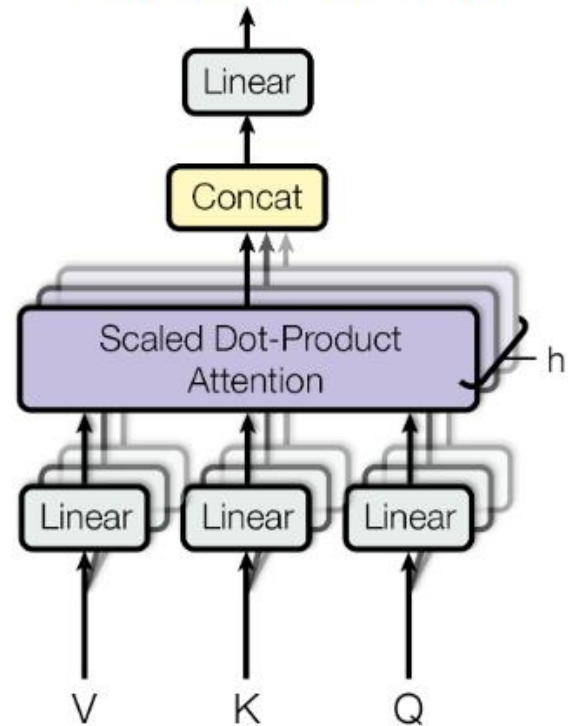
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-head attention

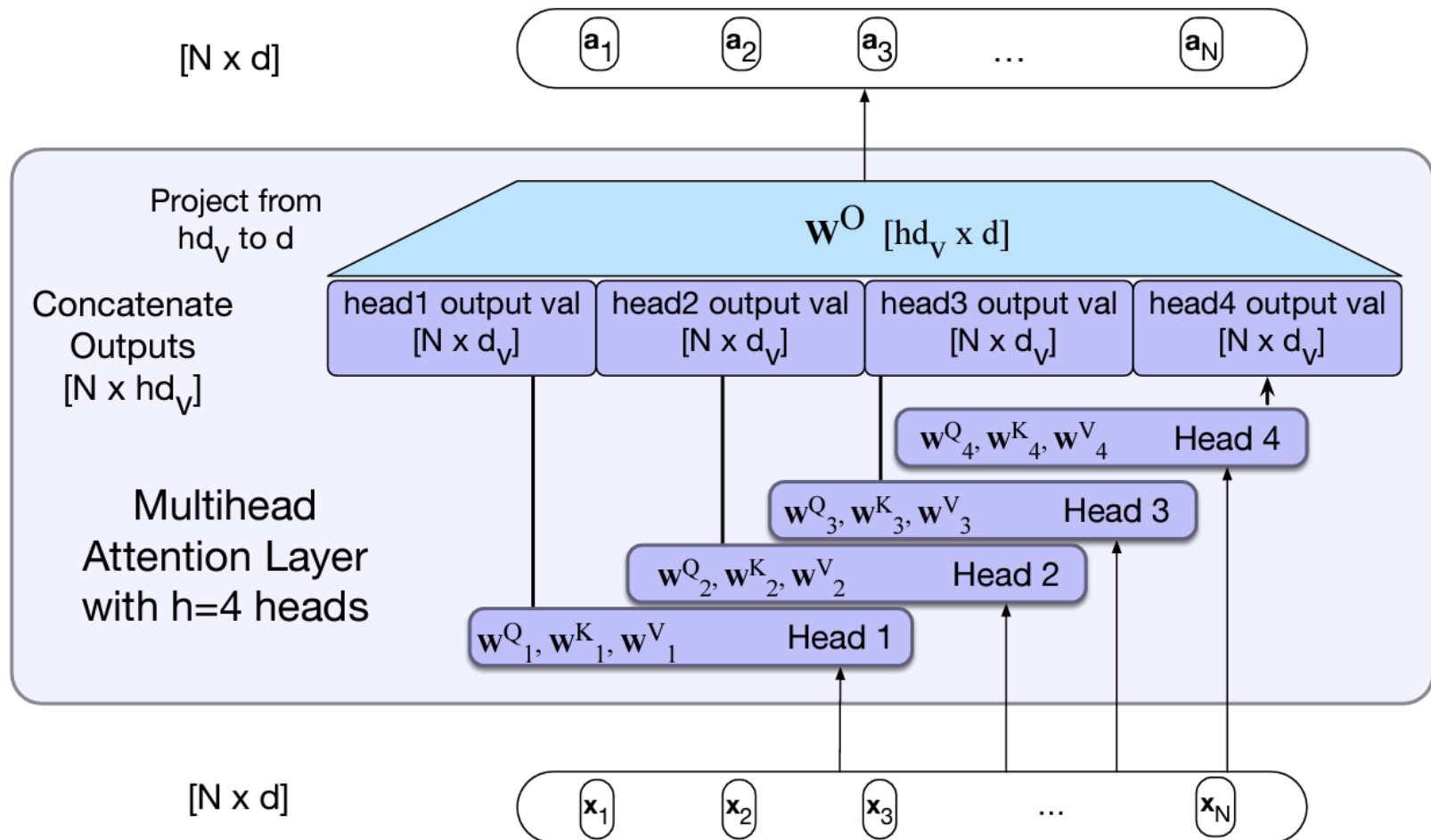
Scaled Dot-Product Attention



Multi-Head Attention



Multi-head attention



Self attention v.s. Cross attention

☐ Self Attention

- ☐ Key, Value, and Query from the same set of tokens

☐ Cross Attention

- ☐ Key, and Value from one set of tokens
- ☐ Query from another set of tokens
- ☐ E.g. words in one language pay attention to words in another.

Quiz

(2 points) Consider a single-headed self-attention mechanism that processes N input tokens of D dimensions to produce N output tokens of the same size. How many weights and biases are used to compute the queries, keys, and values that have D dimensions? What is the size of the attention matrix? If not using attention, how many weights and biases would there be in a fully connected network relating all DN inputs to all DN outputs?

Solution: $3D^2$ weights and $3D$ biases are needed to compute queries, keys and values. Attention matrix is of size $N \times N$. If using fully connected network, there would be $(DN)^2$ weights and DN biases relating DN inputs to DN outputs.

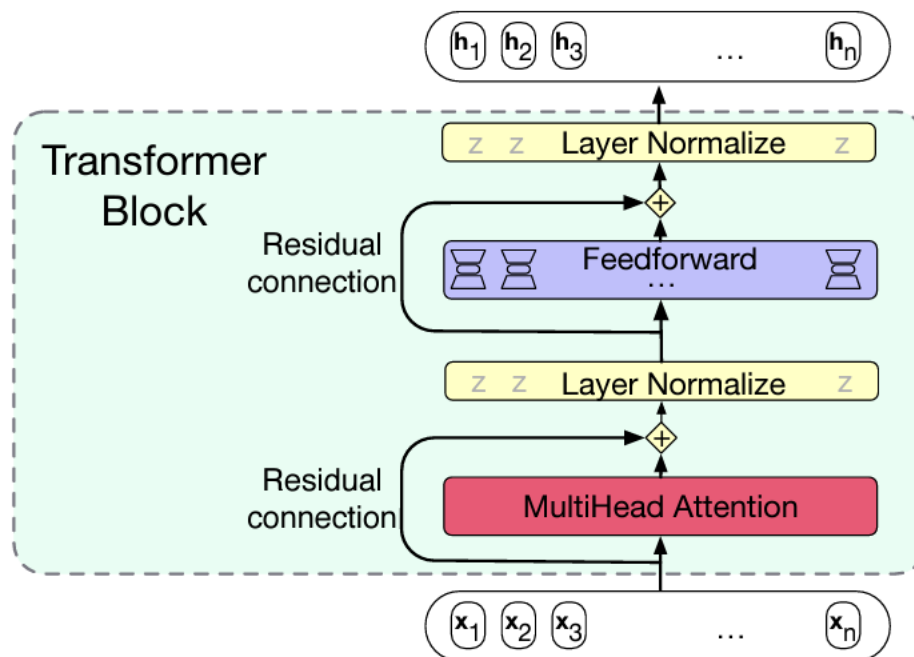
Quiz

(5 points) In self-attention operation, N input tokens are first projected to queries, keys, and values. $N \times N$ attention map depends on

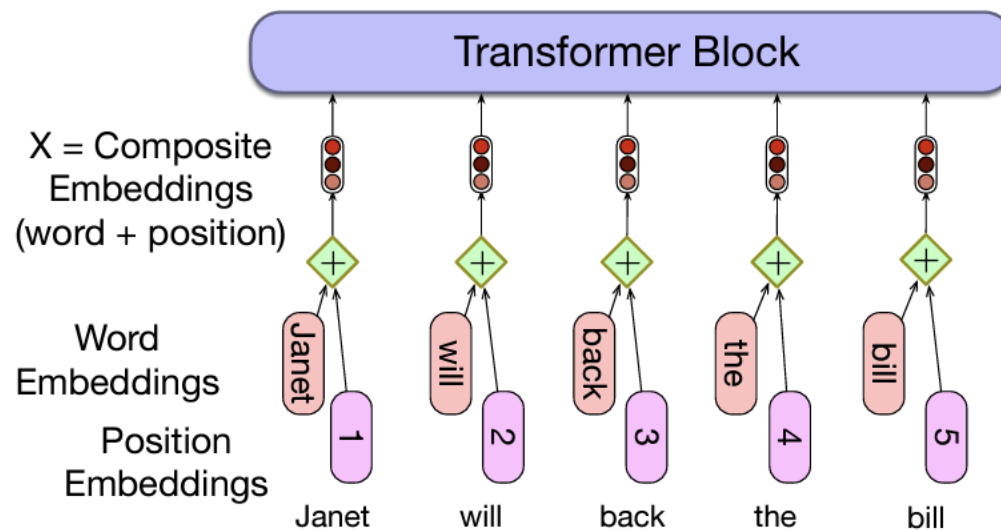
- (a) Value and Key
- (b) Key and Query
- (c) Value and Query
- (d) Query, Key, and Value

From Attention to Transformer Block

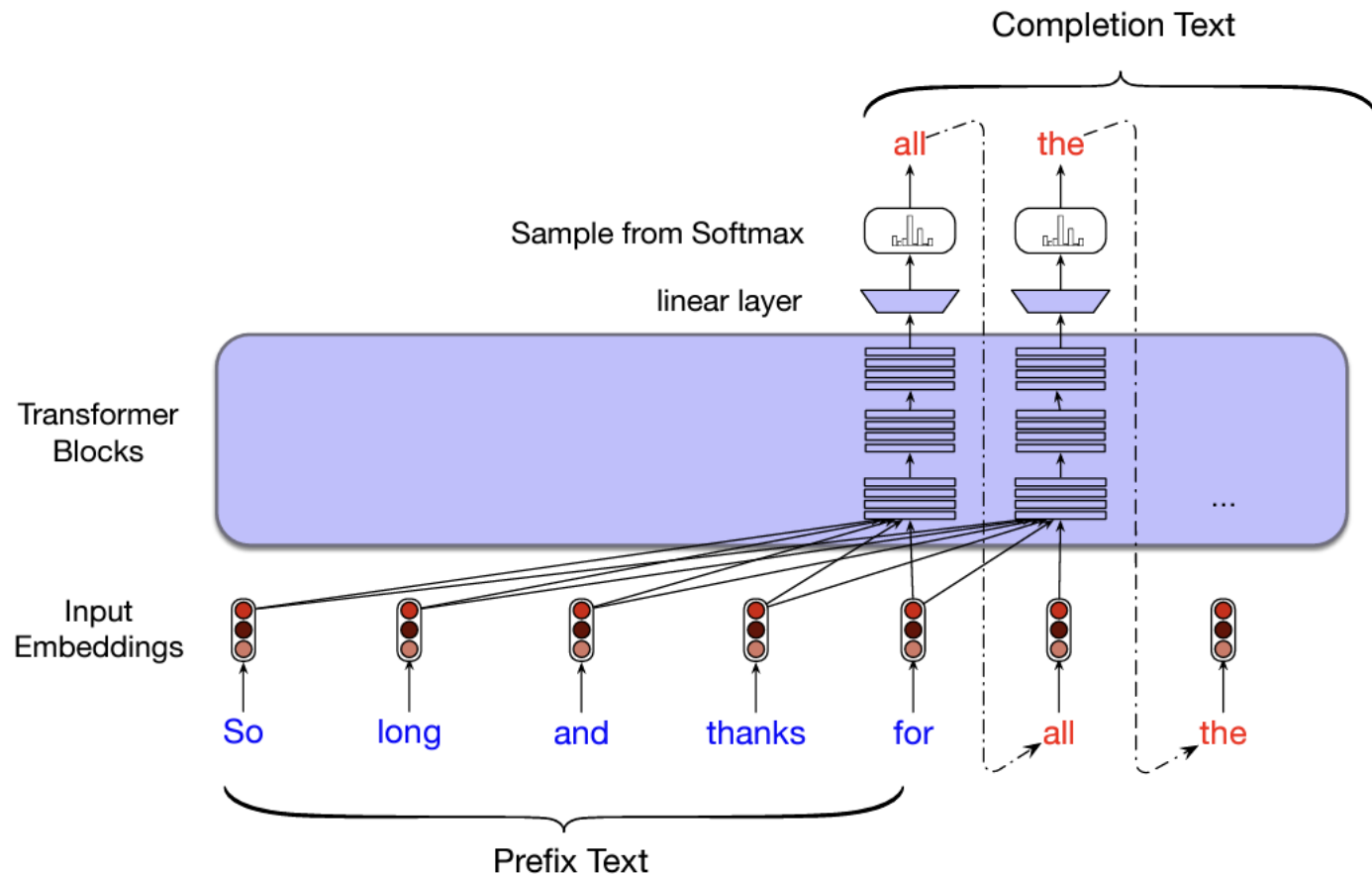
- ❑ A transformer block has
 - ❑ **Self Attention**: information exchange between tokens
 - ❑ **Feed forward network**: Information transform within tokens
 - ❑ E.g. a multi-layer perceptron with 1 hidden layer
 - ❑ **Normalization** (Layer normalization)
 - ❑ **Residual connection**



Embedding for token and position



Transformer-based Large Language Model



AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

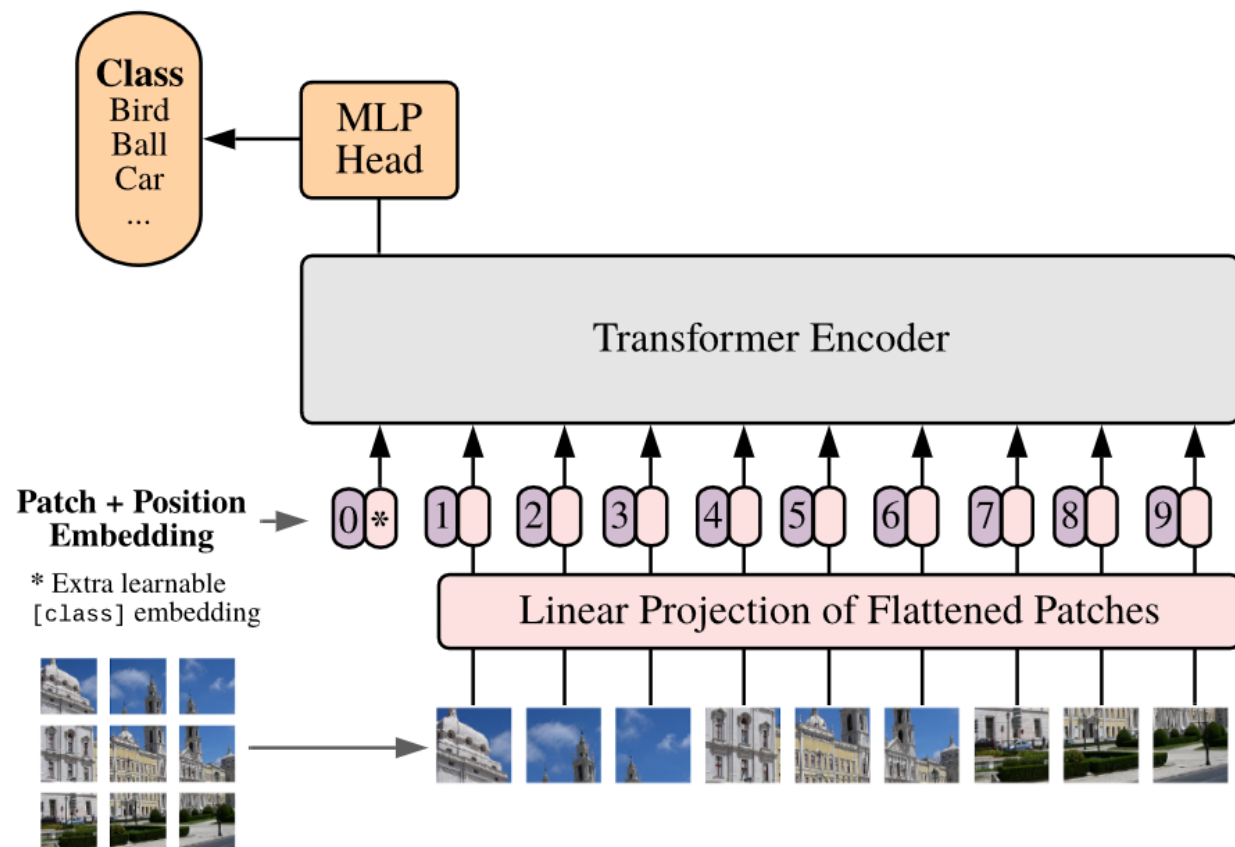
Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train¹

Vision Transformer (ViT)



Transformer Encoder

